

Introduction to Software Engineering

Room4U

Project Proposal & Plan

Team T03

Nguyễn Thành Long (22127245)
Trần Thiên Phúc (24127506)
Lại Minh Thông (24127127)
Nguyễn Kim Ngân (24127461)

Software Engineering Department
Faculty of Information and Technology
University of Science

Table of Contents

1 Objectives **1**

2 Member Contribution Assessment **1**

3 Vision **2**

3.1 Preliminary Problem Statement 2

3.2 Product Position Statement 3

3.3 Stakeholders and Users 3

3.4 Scope and Key Features 4

3.4.1 Core Features 4

3.4.2 Prototype Business Rules and Constraints 4

3.5 Real-World Scenarios 5

3.5.1 Student Books a Room to Practice a Presentation 5

3.5.2 Librarian Handles a Room Requiring Maintenance 6

3.6 Similar Products and Competitors 6

3.7 Differentiation 7

4 Proposed Solution **8**

4.1 Preliminary Architecture 8

4.2 Technology Stack 9

4.3 Hardware and Operating Environment 9

5 Project Plan **10**

5.1 Process and Phase Gates 10

5.2 Work Breakdown Structure 12

5.3 Estimation 16

5.4 Schedule 18

5.5 Roles and Responsibilities 20

5.6 Human Resources and Cost Estimate 20

6 Risk Management **21**

6.1 Risk Register 21

7 Tools & Collaboration Setup **24**

7.1 Tools and Working Rules 24

7.2 Repository Structure 24

7.3 Task Workflow 25

8 AI Usage Log **26**

1 Objectives

This document focuses on the following topics:

- Completing the Project Proposal & Plan document with the following sections:
  - Vision
  - Proposed Solution
  - Project Plan
  - Risk Management
  - Tools & Collaboration Setup
- Including an AI usage log for this phase.
- Understanding the Project Proposal & Plan document.

2 Member Contribution Assessment

ID	Name	Role(s)	Contribution (%)	Evidence
22127245	Nguyễn Thành Long	PM	25%	Issue #1
24127506	Trần Thiên Phúc	Backend Developer	25%	Issue #4
24127127	Lại Minh Thông	BA	25%	Issue #2
24127461	Nguyễn Kim Ngân	Designer	25%	Issue #3

3 Vision

3.1 Preliminary Problem Statement

Room4U addresses the need to reserve group-study rooms in the university library. For this project, the team assumes that the library has eight rooms, each with separate capacity, equipment, and available time slots. Students often need rooms to meet with their groups, practice presentations, or work on assignments together. Before submitting a request, they need to know which rooms are suitable and which time slots are available.

In the illustrative business scenario, receiving requests through in-person conversations or disconnected forms leaves room information and request statuses scattered across different places. Students may submit requests for time slots that have already been booked, while librarians must check schedules before responding. This scenario is an assumption for modeling the problem; it is not a conclusion based on a survey of HCMUS's actual processes.

The product provides a single place for students to view rooms, select a time, submit requests, and track their outcomes. A valid request initially has Pending status. Librarians review the pending requests and decide whether to approve or reject them. A request becomes a confirmed booking only after it has been successfully approved. Students may cancel their bookings according to the published rules.

Librarians need to manage room information, temporarily take rooms out of service for maintenance, and block maintenance time slots. The system supports checks for room capacity, booking duration, opening hours, advance-booking limits, and schedule conflicts. These checks must be enforced by the backend; the user interface only assists with data entry and displays feedback. Concurrent approval actions must not create two approved bookings that overlap for the same room.

The project will deliver a standalone simulation website using test data. Student and Library Administrator accounts will be provided for testing and demonstration. If university email addresses are used, they are test-account data and do not imply integration with the university's single sign-on system. The system will not connect to the student database, a real room-booking system, door locks, or library attendance devices.

Users will access the system through HTML5- and JavaScript-compatible browsers over HTTPS. The planned backend uses Node.js with Express, business data is stored in PostgreSQL, and Firebase Authentication supports user authentication. The system is assumed to run on a Linux server managed by the team. These choices reflect the team's assumed experience with JavaScript and relational databases and must be validated by setting up the environment before implementation begins.

The four-member team will follow the Waterfall model and combine it with test planning based on the V-model. PA1 establishes the preliminary plan; detailed requirements and acceptance criteria will be finalized in PA2, design in PA3, and the test plan in PA4. Must-have features are the implementation commitments for PA5. Changes made after each baseline must be recorded, their impacts assessed, and the relevant updates included in the next PA.

Preliminary acceptance goals are that a student can submit a valid request; a librarian can approve or reject it and the student can see the correct status; and no two Approved bookings can overlap for the same room and time. The release must be deployed and accessible, and include a test report, demonstration video, run instructions, and evidence of team collaboration. Performance targets, test datasets, and exception rules will be clarified in PA2.

3.2 Product Position Statement

Element	Content
For	Students who need group-study rooms and librarians who manage them
Who	Need a single place to view schedules, submit requests, and track their status
The Room4U website	Is a group-study room booking and management website with an approval workflow
That	Centralizes room schedules, requests, and statuses while helping prevent double bookings
Unlike	Separate request forms and independently maintained room schedules
Our product	Combines request submission, rule validation, and librarian approval in one workflow

3.3 Stakeholders and Users

#	Stakeholder / user	Description	Main interests
1	Students	Direct users	Find suitable rooms, submit or cancel requests, and view request statuses
2	Librarians	Direct users	Review requests and manage rooms and availability
3	Assumed library representative	Business sponsor in the illustrative scenario	Clear schedules and room use in accordance with library rules
4	Instructors and teaching assistants	Evaluate the product and course process; not library customers or booking approvers	Assess the project deliverables and process
5	R4U team	Builds, tests, and operates the demo	Keep the project scope achievable within the available time

Planning Assumptions

- The project assumes one library, eight rooms, and up to 500 test accounts. The main demonstration will use two student accounts and one administrator account.
- One representative of a student group submits each request. The system will not manage group membership or synchronize personal calendars.
- Accounts will be pre-created. Self-service registration, password recovery, and institution-wide identity management are not Must-have features.
- One administrator is assumed to be on duty, and booking decisions are made manually. Room4U does not guarantee that the library will respond within a fixed time.
- The team's target for the course is a grade of 9–10; this is a goal, not a guaranteed result.

3.4 Scope and Key Features

Scope adjustment from PA0. To meet the course guidance and keep the workload proportionate to the available project time, the team organized the related functionality into five core features. The administrator interface for configuring booking rules is out of scope; the system still enforces the fixed booking rules. The team approved this adjustment. Long is responsible for notifying and discussing it with the TA through change control; this does not imply TA approval.

3.4.1 Core Features

ID	Core feature	User and scope	Priority (M/S/C/W)
F01	Login and role-based access	Students and admins use pre-created accounts to log in and out. The API enforces roles and data ownership.	Must
F02	Room search and schedule viewing	Students view room listings, photos, capacity, and equipment; filter by group size and date; and see approved or blocked time slots.	Must
F03	Personal request management	Students submit requests with a room, time, and group size; view Pending, Approved, Rejected, or Cancelled status and request history; and cancel their own requests before they start.	Must
F04	Booking request processing	Admins review the queue, approve or reject requests, and cancel approved bookings with a reason. Approval rechecks availability and prevents conflicting bookings.	Must
F05	Room and availability management	Admins add or edit room details, stop accepting new requests for a room, and block maintenance periods. There is no interface for configuring all booking rules.	Must

3.4.2 Prototype Business Rules and Constraints

- The prototype covers one library with eight rooms, pre-provisioned test accounts, and one student representative submitting each group request.
- The agreed fixed rules are opening hours of 08:00–20:00; booking durations of 30–120 minutes; booking up to seven days in advance; and no booking that exceeds room capacity. Time is interpreted in `Asia/Ho_Chi_Minh`. A 30-minute booking-start increment is an assumption for the PA1 planning baseline and will be confirmed and detailed in PA2.
- Pending requests do not reserve a room. The backend rejects approval if an Approved booking or maintenance block overlaps the requested time. Adjacent intervals, such as 09:00–10:00 and 10:00–11:00, are allowed.
- Admins cannot directly change the time or room of an Approved booking. To make a change, they must cancel it with a reason and submit a new request; booking history is retained.

- Before blocking time for maintenance, admins must resolve any affected Approved bookings. Students can see the cancellation reason for their own requests.
- Conflict prevention, capacity checks, and status history support F02–F05; they are not additional core features.

Out of scope:

- Self-service registration and integration with the university's actual login system.
- Payments, AI features, and native mobile applications.
- Email and reminders, or a separate notification center.
- Check-in, analytics, feedback, and no-show penalties.
- An interface for configuring booking rules.

3.5 Real-World Scenarios

The following scenarios illustrate possible use of Room4U. They are not events surveyed or observed at HCMUS and do not describe confirmed university processes.

3.5.1 Student Books a Room to Practice a Presentation

Context: Mai represents a group of four students that needs a room with a display from 14:00 to 15:30 the following day. The group wants confirmation that its request has been approved before going to the library.

1. Mai logs in with a test Student account and filters rooms by a capacity of at least four for the date needed.
2. Mai selects R02, which has six seats and a display, checks its schedule, and submits a request for 14:00–15:30. The system validates the request, creates it with Pending status, and clearly indicates that it is not yet confirmed.
3. A librarian logs in with an Admin account, reviews the request details, and chooses to approve it. The system rechecks the room, schedule, and conflicts before changing the status to Approved.
4. Mai opens her personal requests, sees the Approved status and room details, and the group uses the room during the confirmed period.

Outcome: The student receives clear confirmation, and the room schedule reflects the approved booking. If another request for the same time was approved first, Mai's approval must fail safely. The admin can reject the request with a reason, and Mai can choose another room.

Related features: F01–F04. **Demonstration criteria:** Submitting a request does not reserve the schedule; approval reserves it; and the student sees the correct status.

3.5.2 Librarian Handles a Room Requiring Maintenance

Context: The display in R03 is broken. A librarian needs to take the room out of service from 10:00 to 12:00, but an Approved booking already exists from 10:00 to 11:00.

1. The librarian logs in with an Admin account and checks the affected bookings. The system does not allow a maintenance block that overlaps an Approved booking.
2. The librarian cancels the affected booking and records the maintenance reason. The system retains the booking history and changes its status to Cancelled.
3. The librarian creates a maintenance block for 10:00–12:00. Students see that the room is unavailable during this period, and the system prevents overlapping requests from being created or approved.
4. The affected student opens their personal requests, sees the status and cancellation reason, and can search for R04 and submit a new request if it is suitable.

Outcome: The broken room is not assigned to another group, and the cancellation is reasoned and traceable. The application does not automatically move the booking to another room or send email.

Related features: F01–F05.

3.6 Similar Products and Competitors

The team compared three real products using their official documentation, accessed October 16, 2026. The Room4U assessments below express the team's positioning; they are not benchmark results or findings from comparative testing.

Solution	Capabilities confirmed by documentation	Room4U positioning in comparison
Google Calendar in Google Workspace [1]	A resource manager can accept or decline events scheduled for a room or other resource after an administrator configures resource management.	Room4U focuses on a dedicated study-group request interface and business statuses. It does not integrate with personal calendars or Google Workspace.
Springshare LibCal [2]	Provides library room reservations, booking mediation, and check-in, as well as event calendars and other library services.	Room4U is scoped to five core features for one simulated library; it does not aim to provide a broad library-services suite like LibCal.
MRBS Meeting Room Booking System [3]	A free, GPL web application for booking rooms or other resources. Its documented features include conflict prevention, multiple authorization levels, and day, week, and month calendar views.	Room4U is positioned around a Vietnamese-language workflow and the rules selected for this course project. It does not aim to match MRBS's feature breadth.

3.7 Differentiation

Room4U's proposed differentiation lies in its scope and organization of the user experience: one library, two roles, five core features, and a clear group-study request workflow. The planned Vietnamese interface will use concise labels corresponding to Pending, Approved, Rejected, and Cancelled, and show the applicable rules and processing reasons alongside each request. This is a design choice to validate with students, not evidence that Room4U is easier to use than competing products.

Compared with a general-purpose calendar, Room4U presents room use as a request for librarian review. Compared with broader room-booking platforms, the team deliberately limits the product to a defined set of rules and the operations needed for the modeled group-study library. The goal is to make the product feasible for a four-member team to specify, build, and test end to end.

Manual approval, conflict prevention, and online booking already exist in current solutions; Room4U does not claim them as inventions or exclusive features [1–3]. Its proposed differentiation is the combination of scope, context, and interface design for the target users. The practical value of this combination must be evaluated after a prototype is available.

References

Official product documentation accessed October 16, 2026.

[1] Google Workspace Admin Help, "Approve or deny Calendar room & resource bookings," https://knowledge.workspace.google.com/admin/calendar/approve-or-deny-calendar-room-and-resource-bookings.

[2] Springshare, "LibCal for Academic Libraries," https://www.springshare.com/academic-libraries/libcal.

[3] MRBS, "Introduction," https://mrbs.sourceforge.io/.

[4] Instructor-provided project idea guidance, https://pub.nndkhoa9.win/intro2se/project-idea.png.

4 Proposed Solution

4.1 Preliminary Architecture

Figure 1 shows the C4 System Context diagram of Room4U. Students use Room4U to browse rooms, submit booking requests and track their status. Library administrators use it to approve or reject requests, cancel approved bookings and manage rooms and maintenance periods. Firebase Authentication is the only external system: it verifies user identities, while Room4U itself checks each user's role and permissions and stores all business data. The database is an internal part of Room4U and therefore does not appear as an external system at this level. Containers, components and technical mechanisms are detailed in PA3.

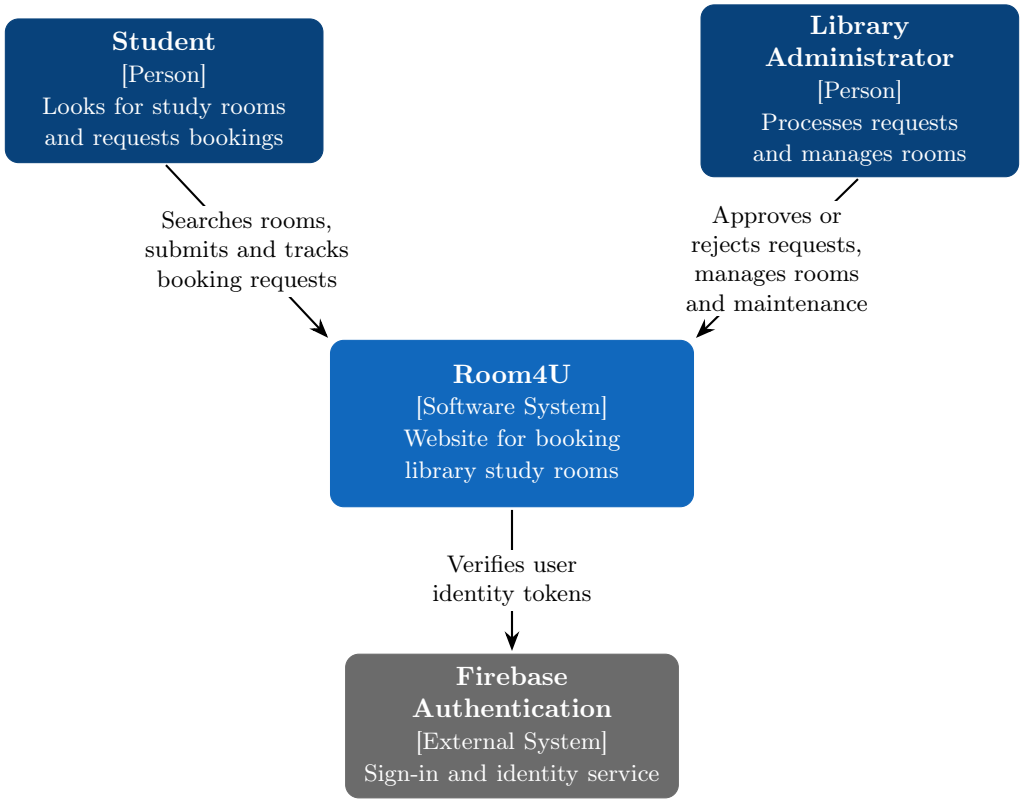


Figure 1: C4 System Context diagram of Room4U. The database is an internal part of Room4U and is therefore not drawn as an external system.

Preliminary business rules. The rules below are working assumptions used for scoping and estimation. They will be confirmed and refined in PA2.

- The library opens from 08:00 to 20:00 every day. All times use the `Asia/Ho_Chi_Minh` time zone.
- A booking starts on a 30-minute boundary (a PA1 planning assumption to be confirmed and detailed in PA2), lasts between 30 and 120 minutes, and can be made at most 7 days in advance.
- A Pending request does not hold a room. When an administrator approves a request, the back end re-checks availability, and concurrent approvals are guarded by a transaction or locking mechanism designed in PA3.

- Two time intervals $[start, end)$ conflict when $start_1 < end_2$ and $start_2 < end_1$. A booking that ends at 10:00 therefore does not conflict with one that starts at 10:00.
- A student may cancel their own Pending request or Approved booking before its start time. Rejected and Cancelled bookings do not occupy the room.
- An administrator cannot create a maintenance period that overlaps an Approved booking; the affected bookings must first be cancelled with a reason. A suspended room does not accept new requests.

4.2 Technology Stack

Table 1: Technology stack of Room4U.

Layer	**Technology**	**Reason for choice**
Front end	HTML, CSS, JavaScript	Suitable for a demo with a small number of screens; runs in any modern browser without extra tooling.
Back end	Node.js with Express	Same language as the front end; a layered structure (routes, services, repositories) keeps the business rules in one place.
Database	PostgreSQL	Rooms, users and bookings are relational data; transactions and constraints help prevent double booking.
Authentication	Firebase Authentication	Handles sign-in for pre-created accounts; the back end verifies tokens and checks the roles stored in the business database.
Hosting and deployment	Linux VPS with Nginx	Runs the Node.js back end and PostgreSQL; HTTPS, with configuration kept in environment variables.
Testing and CI	Vitest, Supertest, Playwright, GitHub Actions	Unit, API, and end-to-end tests are planned. Initial GitHub Actions CI setup is planned for W8, with checks added as implementation proceeds.
Source control and tasks	GitHub, GitHub Projects	GitHub Projects tracks WBS-linked tasks; the agreed workflow requires pull-request review before merge.

The stack is a planning decision rather than a final architecture choice. During the design phase (Weeks 6–7) the team will validate it with a short environment spike: running the front end, back end and database together on a development machine, verifying a Firebase ID token in the Express back end, and making a trial deployment to a Linux server. If any step fails, the alternative will be recorded as an Architecture Decision Record in PA3. Secrets such as API keys and service-account files will never be committed to the repository.

4.3 Hardware and Operating Environment

Table 2 lists the environments needed to develop, test and demonstrate Room4U.

Table 2: Development, test and demonstration environments.

Environment	Planning assumption
Development machines	Laptops running Windows, macOS or Linux with at least 8 GB of RAM, Git, Visual Studio Code and one agreed Node.js LTS version.
Demo server	Linux server with 2 vCPUs, 4 GB of RAM and 20 GB of storage, hosting one back end instance and one database. These figures are assumptions and will be checked with a smoke test.
User devices	Laptops or phones with a modern browser. Testing focuses on Chrome and Edge on desktop, plus one mobile viewport.
Test data	Eight simulated rooms; pre-created Student and Administrator accounts; data sets covering schedule conflicts, time limits, capacity limits and permissions.
Demo operation	HTTPS, Internet access, a Firebase test project, a data backup before the demo, and a backup video in case the network fails.

5 Project Plan

5.1 Process and Phase Gates

The project follows the waterfall model, with test planning prepared in the spirit of the V-model: the PA4 test plan is derived from the PA2 requirements and the PA3 design. Each phase ends with a phase gate, which is a PA submission. A submitted PA becomes a baseline; any later change is recorded, its impact is assessed, and it is listed in the next PA. Functional coding is planned to start after the PA3 gate.

Table 3: Phase gates and exit criteria.

Gate	Phase	Planned due	Exit criteria
PA1	Initiation and planning	End of Week 3	Vision, scope, and the approved WBS baseline, estimates, schedule, and risks are consistent. GitHub Projects lists all current work packages, owners, and deadlines. Before PA1 submission, configure and verify main-branch protection and at least one required PR approval, grant and verify instructor/TA repository access, and confirm TA participation in Discord.
PA2	Requirements	End of Week 5	Functional and non-functional requirements, business rules and acceptance criteria are clear and prioritised; use cases and prototype are complete; the traceability matrix and requirements review are finished.
PA3	Design	End of Week 7	C4 diagrams, UML models, ERD and UI design are consistent with each other and with PA2; at least three ADRs are recorded; the design covers all Must-have requirements; major review findings are closed.
PA4	Test planning	End of Week 8	The test plan applies equivalence partitioning, boundary value analysis and decision tables; test cases trace to every Must-have requirement; the test environment and pass/fail criteria are defined.
PA5	Implementation, testing and deployment	End of Week 10	All Must-have features work and are deployed; automated tests cover at least two use cases with at least two scenarios per use case; test and coverage reports, CI, code-review evidence, working-software video, and user guide are available.
Final	Presentation and demo	Week 11	A rehearsal of the demo succeeds; the slides are complete; every member speaks for at least 2 minutes and can explain their own work.

Week numbers are the planning baseline; official submission dates and times are those published on Moodle.

Change control. Anyone may propose a change by opening a change-request (CR) issue that states the reason, the affected requirements, the estimated effort and the impact on the schedule. The business analyst assesses the business impact, a developer assesses the technical impact, and the team lead consolidates the results. The whole team decides; changes to the committed scope are discussed with the TA. After approval, the documents, task board and traceability matrix are updated, and the change is listed in the next PA.

5.2 Work Breakdown Structure

The WBS has three levels. Level 1 consists of the project phases, level 2 of deliverables or groups of deliverables, and level 3 of work packages. For example: 5 Implementation, testing and deployment → 5.1 Back end core and infrastructure → 5.1.2 Booking and double-booking prevention. Each work package has exactly one owner, an effort estimate in person-hours and a target week, and is tracked as one issue on the task board. WBS identifiers are independent of the section numbers of this document.

Table 4: WBS levels 1 and 2.

Level 1 (phase)	**Level 2 (deliverable groups)**
1 Planning (PA1)	1.1 Vision; 1.2 Solution and work breakdown; 1.3 Estimation and schedule; 1.4 Risk, tools and submission
2 Requirements (PA2)	2.1 Requirements specification; 2.2 Use cases and prototype; 2.3 Review and baseline
3 Design (PA3)	3.1 Architecture and data; 3.2 Object-oriented and UI design; 3.3 Review and packaging
4 Test planning (PA4)	4.1 Test design; 4.2 Review and environment
5 Implementation, testing and deployment (PA5)	5.1 Back end core and infrastructure; 5.2 Front end; 5.3 Admin API and integration; 5.4 System testing and reporting
6 Final presentation	6.1 Presentation
7 Continuous management	7.1 Management throughout the project

Effort values are expert-judgement estimates in person-hours; further estimation details are provided in Section 5.3. The effort of each phase already includes its reviews and its business and progress meetings. Continuous management (group 7) covers only the consolidation of weekly reports and the maintenance of the task board and repository, so no effort is counted twice.

Table 5: Work packages for PA1 and PA2.

WBS	**Work package**	**Owner**	**Hours**	**Week**
1	**Planning (PA1)**			
1.1 Vision				
1.1.1	Vision and scope	Long	4	W3
1.1.2	Stakeholders and product positioning	Long	2	W3
1.2 Solution and work breakdown				
1.2.1	Context diagram, technology stack and environment	Thông	4	W3
1.2.2	Phase gates and WBS	Thông	3	W3
1.3 Estimation and schedule				
1.3.1	Estimation, Gantt schedule, roles and cost	Ngân	5	W3
1.4 Risk, tools and submission				
1.4.1	Risk register	Phúc	2	W3

WBS	Work package	Owner	Hours	Week
1.4.2	Repository, task board and communication setup	Phúc	2	W3
1.4.3	Assembly, cross-review and submission of PA1	Long	2	W3
Subtotal PA1			**24**	
2 Requirements (PA2)				
2.1 Requirements specification				
2.1.1	Functional requirements and business rules	Thông	7	W4–5
2.1.2	Non-functional requirements and acceptance criteria	Long	4	W4–5
2.2 Use cases and prototype				
2.2.1	Use cases and exception flows	Thông	5	W4–5
2.2.2	Prototype of the main flows	Ngân	8	W4–5
2.3 Review and baseline				
2.3.1	Traceability matrix and SRS review	Phúc	6	W5
2.3.2	Assembly and baseline of PA2	Long	6	W5
2.3.3	Prototype review	Ngân	2	W5
2.3.4	Data feasibility check	Phúc	2	W4
Subtotal PA2			**40**	

Table 6: Work packages for PA3 and PA4.

WBS	Work package	Owner	Hours	Week
3 Design (PA3)				
3.1 Architecture and data				
3.1.1	C4 architecture and ADRs	Phúc	6	W6
3.1.2	ERD and table specifications	Phúc	6	W6–7
3.2 Object-oriented and UI design				
3.2.1	UML models and class specifications	Thông	8	W6–7
3.2.2	UI design and screen flow	Ngân	8	W6–7
3.3 Review and packaging				
3.3.1	Traceability update and design review	Long	6	W7
3.3.2	Assembly of PA3	Ngân	2	W7
3.3.3	Final check and submission of PA3	Long	2	W7
3.3.4	Testability review	Thông	2	W7
Subtotal PA3			**40**	
4 Test planning (PA4)				
4.1 Test design				

WBS	Work package	Owner	Hours	Week
4.1.1	Test plan, equivalence partitioning, boundary values, decision tables	Thông	4	W8
4.1.2	UI test cases	Ngân	2	W8
4.2 Review and environment				
4.2.1	Review and submission of PA4	Long	1	W8
4.2.2	Test environment and test traceability	Long	1	W8
Subtotal PA4			**8**	

The 8 hours of PA4 reuse the specifications and acceptance criteria produced in PA2; the execution of system tests is counted under group 5.4.

Table 7: Work packages for PA5, the final presentation and continuous management.

WBS	Work package	Owner	Hours	Week
5 Implementation, testing and deployment (PA5)				
5.1 Back end core and infrastructure				
5.1.1	Database and authentication (F01)	Phúc	9	W8
5.1.2	Booking and double-booking prevention (F03, F04); review the PA3 booking API contract and acceptance criteria in W8, then implement booking and conflict prevention in W9	Phúc	10	W8–9
5.1.3	Back end unit tests and CI; initial CI setup in W8, with unit tests and coverage checks completed in W10	Phúc	5	W8–10
5.1.4	Deployment and smoke test	Phúc	3	W10
5.2 Front end				
5.2.1	Room list and calendar (F02)	Ngân	5	W8
5.2.2	Submit, view and cancel requests (F03)	Ngân	7	W8–9
5.2.3	Administrator screens (F04, F05); prioritize approval UI for W9 integration and complete room/maintenance management in W10	Ngân	8	W9–10
5.2.4	Front end unit tests	Ngân	3	W9–10
5.3 Admin API and integration				
5.3.1	Approval and room-management API	Long	6	W8–9
5.3.2	Integrate ready student-booking and admin-approval flows in W9; integrate room/maintenance management after its UI and API are complete in W10	Long	5	W9–10
5.3.3	Defect fixing after system testing	Long	4	W10

WBS	Work package	Owner	Hours	Week
5.3.4	Prepare document structure and a guide draft from available flows in W9; update against the tested deployed build and record the video in W10	Long	5	W9–10
5.4 System testing and reporting				
5.4.1	Test data and test accounts	Thông	4	W8
5.4.2	Prepare and execute automated tests for at least two use cases, with at least two scenarios per use case, after the corresponding workflows are integrated	Thông	6	W8–10
5.4.3	Test integrated workflows in W9; complete system testing and retesting in W10	Thông	9	W9–10
5.4.4	Prepare the report structure and collect available results in W9; finalize after W10 test results	Thông	5	W9–10
Subtotal PA5			**94**	
6 Final presentation				
6.1 Presentation				
6.1.1	Presentation script and overview slides	Long	3	W11
6.1.2	Business demo and Q&A	Thông	3	W11
6.1.3	UI slides and rehearsal	Ngân	3	W11
6.1.4	Demo check and backup	Phúc	3	W11
Subtotal Final			**12**	
7 Continuous management				
7.1 Management throughout the project				
7.1.1	Coordination and weekly reports	Long	9	W2–10
7.1.2	Board updates for business work	Thông	3	W2–10
7.1.3	Board updates for UI work	Ngân	3	W2–10
7.1.4	Repository and technical progress updates	Phúc	3	W2–10
Subtotal continuous management			**18**	

Planned scenarios for WBS 5.4.2. This is proposed test scope, not a report of completed tests: submit booking request (valid request; request outside opening hours) and approve booking request (available room/time; conflicting booking, including the concurrent-approval case).

Table 8: Effort summary by phase (baseline, before contingency).

Phase	Effort (person-hours)
1 Planning (PA1)	24
2 Requirements (PA2)	40
3 Design (PA3)	40
4 Test planning (PA4)	8
5 Implementation, testing and deployment (PA5)	94
6 Final presentation	12
7 Continuous management	18
Total baseline effort	**236**

Definition of done. A work package is complete when:

- the deliverable or code change described in its issue exists and its acceptance criteria are met;
- the designated reviewer has checked it, the documents or code are updated in the repository through a pull request, and no blocking comment remains;
- the owner has recorded the actual effort, the issue status, the link to the pull request or document, and any remaining issues on the task board.

Early technical checkpoints.

- F03 and F04 share the same status model but are used by different roles. The sequence diagrams must show the permission check and the conflict check at approval time.
- The ERD and the transaction design must support rooms, bookings, user roles, maintenance periods, the fixed booking rules and the status history.
- Test design must cover time boundaries, capacity limits, bookings outside opening hours, cancellations close to the start time and concurrent approvals.
- Candidate ADRs for PA3 are the choice of PostgreSQL, the use of Firebase for authentication, and the mechanism that prevents two Approved bookings from overlapping. PA1 does not replace these ADRs.

5.3 Estimation

The team used bottom-up estimation and expert judgement. Package owners estimated the work packages, and the team reviewed and approved the work packages, owners, and estimates as the PA1 planning baseline. Effort is measured in person-hours, not calendar duration; estimates include only the work and review/test activities specified in each package, avoiding double-counting. The approved estimated-effort baseline is 236 person-hours: PA1 24, PA2 40, PA3 40, PA4 8, PA5 94, Final 12, and continuous management 18. These are estimates, not actual hours worked. Changes after this baseline follow change control.

Planned work-package allocation for W8–W10. The table gives the planned weekly distribution in person-hours. Allocations for PA4 and PA5 packages sum to each package's full WBS effort. Rows marked with an asterisk show only the W8–W10 share of the existing continuous-management packages.

Table 9: Planned W8–W10 work-package allocation (person-hours).

WBS ID	Owner	W8	W9	W10
4.1.1	Thông	4	0	0
4.1.2	Ngân	2	0	0
4.2.1	Long	1	0	0
4.2.2	Long	1	0	0
5.1.1	Phúc	9	0	0
5.1.2	Phúc	0.25	9.75	0
5.1.3	Phúc	0.5	0	4.5
5.1.4	Phúc	0	0	3
5.2.1	Ngân	5	0	0
5.2.2	Ngân	2.5	4.5	0
5.2.3	Ngân	0	4.5	3.5
5.2.4	Ngân	0	0.5	2.5
5.3.1	Long	3	3	0
5.3.2	Long	0	3	2
5.3.3	Long	0	0	4
5.3.4	Long	0	2	3
5.4.1	Thông	4	0	0
5.4.2	Thông	1	4	1
5.4.3	Thông	0	3	6
5.4.4	Thông	0	2.5	2.5
7.1.1*	Long	1	1	1
7.1.2*	Thông	0.25	0.25	0.5
7.1.3*	Ngân	0.25	0.25	0.5
7.1.4*	Phúc	0.25	0.25	0.5
W8–W10 total		**35**	**38.5**	**34.5**

The asterisked entries are the W8–W10 portions only; the full WBS effort remains 9 hours for 7.1.1 and 3 hours for each of 7.1.2–7.1.4. These allocations are planning figures, not actual effort or evidence of completed work. The W8–W10 workload totals 108 hours: 94 for PA5, 8 for PA4, and 6 from existing continuous-management effort.

The usual planning allowance is about 7 hours per person per week. The team has agreed to make 10 hours per person per week available in W8–W10, for a confirmed capacity of $4 \times 10 \times 3 = 120$ person-hours. Long has confirmed enough availability for his assigned W3 and W7 work, including continuous-management tasks; the usual 7-hour allowance is not a hard limit for his workload in those weeks.

Table 10: W8–W10 planned effort and capacity by member (person-hours).

Member	PA5	PA4	Management W8–W10	Total	Capacity
Long	20	2	3	25	30
Thông	24	4	1	29	30
Ngân	23	2	1	26	30
Phúc	27	0	1	28	30
Total	**94**	**8**	**6**	**108**	**120**

Table 11: W8–W10 planned effort by member and week (person-hours).

Member	W8	W9	W10	Total
Long	6	9	10	25
Thông	9.25	9.75	10	29
Ngân	9.75	9.75	6.5	26
Phúc	10	10	8	28
Total	**35**	**38.5**	**34.5**	**108**

The 12-hour difference between the agreed 120-hour capacity and the 108-hour workload is capacity headroom, not additional effort; the project baseline remains 236 hours. Headroom is 5 hours for Long, 1 for Thông, 4 for Ngân, and 2 for Phúc. W9 and W10 both include members at or near 10 hours, so the PM should track per-member progress and handoffs rather than rely only on group capacity.

5.4 Schedule

The schedule is a week-level planning baseline; official submission dates and times are those published on Moodle. Tasks may overlap where dependencies allow. The W8–W10 package allocations in Table 9 provide the detailed weekly plan; the Gantt below summarizes those spans.

Table 12: Project schedule by course week.

Task / milestone	2	3	4	5	6	7	8	9	10	11
Scope and PA1 planning	■	■								
PA1 submission		♦								
Requirements and acceptance criteria			■	■						
PA2 submission				♦						
Architecture, database, and UI design					■	■				
PA3 submission						♦				
PA4 test plan and environment (4.1–4.2)							■			
PA4 submission							♦			
Database and authentication (5.1.1)							■			
Booking backend preparation and implementation (5.1.2)							■	■		
Room and booking-request UI (5.2.1–2)							■	■		
Admin API (5.3.1)							■	■		
Initial CI configuration (5.1.3)							■			
Admin screens (5.2.3)								■	■	
Front end–back end integration (5.3.2)								■	■	
Automation (5.4.2)							■	■	■	
Front end unit tests (5.2.4)								■	■	

Task / milestone	2	3	4	5	6	7	8	9	10	11
Back end unit tests and coverage completion (5.1.3)									■	
System testing and retesting of integrated workflows (5.4.3)								■	■	
Test report and retrospective (5.4.4)								■	■	
Defect fixing after system testing (5.3.3)									■	
Deployment and smoke test (5.1.4)									■	
W9 documentation draft; W10 final materials/video (5.3.4)								■	■	
Continuous management (7.1.1–4)	■	■	■	■	■	■	■	■	■	
PA5 submission and final review									♦	
Final presentation / demonstration										♦

Legend: Column numbers indicate course weeks; ■ = planned work; ♦ = milestone. The WBS allocation table gives exact weekly effort where a workstream spans multiple weeks.

Dependencies and sequencing:

- Finalize the database schema and API contracts from PA3 before implementation.
- Complete database and authentication work before integrating workflows that require sign-in.
- Integrate incrementally in W9–W10: connect student-booking and admin-approval flows when ready in W9, then connect room and maintenance management after the corresponding UI and API are complete in W10.
- Prepare test cases and automation in advance, but run workflow tests only on flows that have been integrated. In W10, complete the room/maintenance UI and API, integrate them, run the corresponding system tests, fix defects and retest, then verify the release.
- Finalize the test report and PA5 video using the deployed build and results from completed checks.

Expected critical path. Under the current week-level planning assumptions, the expected sequence is requirements baseline (PA2) → architecture, database design, and API contracts (PA3) → database/authentication and core booking backend (5.1.1–5.1.2) → front-end–back-end integration (5.3.2) → system testing, defect fixing, and retesting (5.4.3, 5.3.3) → deployment and smoke testing (5.1.4) → final PA5 report, release documentation, and video (5.4.4, 5.3.4) → final demonstration. Delayed required inputs can postpone integration, testing, and handover. Integration is planned in parts across W9 and W10. Room/maintenance UI and API work must be complete and integrated before full system testing of those functions; in W10, sequence their completion, integration, testing, defect fixing/retesting, and release verification by these prerequisites. The final report and video use the deployed build after checks are complete. Work may overlap partially when its prerequisites are available. This is an expected path, not a CPM or float calculation; confirm it when dependencies and durations are detailed.

5.5 Roles and Responsibilities

The responsibilities below follow the team's agreed role allocation. All members share responsibility for reviewing the report, testing the integrated application, communicating progress, and recording their own contributions.

Role	Main	Backup	Responsibilities
Team Lead / Project Manager	Long	Thong	Maintain the schedule and task board, assign and follow up tasks, organise weekly status meetings, track risks, and coordinate submissions and communication.
Business Analyst / Requirements	Thong	Long	Gather and document requirements, booking rules, use cases, and acceptance criteria; check consistency across PA documents and coordinate requirement reviews.
Backend Developer / Database / Architecture	Phuc	Ngan	Lead C4 and architecture work, ADRs, and ERD; design and maintain the database and backend; implement authentication, role checks, booking validation, and API endpoints.
Frontend Developer / UI Designer	Ngan	Phuc	Design and implement Student and Administrator screens; display room availability and booking statuses; connect the interface to backend APIs and check usability.
Testing and Quality Coordination	Thong	Long	Lead test planning, test data, workflow and integration testing, defect tracking, retesting, and test evidence/reporting; all members support testing as assigned.

5.6 Human Resources and Cost Estimate

The project is planned for a four-member student team. The baseline workload is 236 person-hours according to the WBS. For a comparable cost estimate, the team uses a notional rate of VND 100,000 per person-hour. This rate is a planning assumption to represent the value of effort, not an actual payment to team members. Hosting and contingency figures are provisional and should be revised once the deployment approach is known.

Cost item	Estimated cost (VND)
Human effort: 236 person-hours × VND 100,000/hour	23,600,000
Development tools and collaboration software (free/student tiers)	0
Hosting / deployment allowance for the course demonstration	500,000
Other expenses / contingency allowance	500,000
Total estimated project cost	**24,600,000**

The 500,000 VND contingency is a monetary reserve, separate from capacity headroom and not an additional labor estimate. The estimate assumes that the team uses available free or student plans for source control, documentation, and development. It excludes the purchase of personal computers, existing internet access, and actual salary payments. The team will review the estimate if the feature scope, hosting option, or course requirements change.

6 Risk Management

Probability and Loss are converted using the lecture scales below. Calculate risk exposure (RE) by multiplying the numeric scores directly.

$$\text{RE} = \text{Probability} \times \text{Loss}.$$

Probability	Score	Loss	Score
Unlikely	10	Negligible	10
Seldom	25	Insignificant	25
Occasional	50	Moderate	50
Likely	75	Serious	75
Frequently	90	Catastrophic	90

6.1 Risk Register

ID	Risk	Type	Probability	Loss	RE	Mitigation / Contingency	Owner
R1	Concurrent approvals may create overlapping bookings and disrupt the core reservation workflow.	Technology	Occasional (50)	Catastrophic (90)	4,500	**Mitigation:** Design conflict checks in PA3; specify concurrent-approval tests in PA4 and execute them during PA5. **Contingency:** Disable faulty approval; fix and retest before PA5 submission.	Phúc
R2	If a key team member is unavailable during PA5, Must-have features may remain incomplete by the PA5 submission deadline.	People	Occasional (50)	Catastrophic (90)	4,500	**Mitigation:** Confirm weekly availability; assign a backup; hand over code and documentation regularly. **Contingency:** Reassign tasks to the backup, revise the schedule, and notify the TA if needed.	Long

ID	Risk	Type	Probability	Loss	RE	Mitigation / Contingency	Owner
R3	Changes to booking, cancellation, or approval rules after baseline may force redesign, recoding, and retesting.	Requirements	Occasional (50)	Serious (75)	3,750	**Mitigation:** Review booking rules and exceptions in PA2; document the agreed requirements baseline. **Contingency:** Raise a change request; estimate schedule and effort impact; update the baseline after approval.	Thông
R4	Actual effort may exceed estimates, leaving insufficient time for integration, testing, and deployment.	Estimation	Likely (75)	Serious (75)	5,625	**Mitigation:** Decompose WBS packages; review estimates; compare actual effort with estimates weekly. **Contingency:** Under the team's approved trigger, when a work package's forecast total effort (actual effort to date plus estimated remaining effort) exceeds 120% of its approved estimate, the PM assesses dependencies, remaining capacity, and milestone impact, then reallocates work or raises a change request.	Long
R5	Inconsistent API contracts or booking states may break front-end and back-end integration.	Technology	Occasional (50)	Moderate (50)	2,500	**Mitigation:** Agree on API contracts and sample data in PA3; integrate workflows incrementally. **Contingency:** Align interfaces to the agreed contract and add regression tests for affected workflows.	Ngân

ID	Risk	Type	Probability	Loss	RE	Mitigation / Contingency	Owner
R6	Incorrect hosting, authentication, or environment settings may make the application inaccessible after deployment.	Technology	Occasional (50)	Serious (75)	3,750	**Mitigation:** Validate hosting, authentication, and Firebase settings early; trial deployment in W8–W9. **Contingency:** Restore verified configuration or backup; smoke-test application access and update deployment instructions.	Phúc
R7	Missing tasks, reviews, or weekly reports may weaken evidence of professional team collaboration.	Organizational	Seldom (25)	Serious (75)	1,875	**Mitigation:** Review task-board activity, pull requests, and weekly reports; remind owners before deadlines. **Contingency:** Record gaps accurately, add available evidence, and correct the process the following week.	Long
R8	Late testing may uncover critical defects when too little time remains for fixes.	Technology	Occasional (50)	Serious (75)	3,750	**Mitigation:** Prepare test data and automation from W8; run unit and workflow tests incrementally. **Contingency:** Freeze feature work; fix Must-have blockers first and complete retesting before PA5 submission.	Thông

Prioritize R4 (5,625), followed by R1 and R2 (4,500 each), R3, R6, and R8 (3,750 each), R5 (2,500), and R7 (1,875). Owners should review indicators and mitigation weekly, update the task board, and execute contingency actions with a scope and schedule reassessment if a risk occurs; a lower RE is not a reason to ignore a risk.

7 Tools & Collaboration Setup

The tools below distinguish active team practices from access and controls that remain to be completed before PA1 submission.

7.1 Tools and Working Rules

Tool	Purpose / Working Rules
Moodle	Check course announcements and submit PA deliverables as instructed; Moodle is the required course platform.
Discord	The team uses Room4U Discord; the TA has not joined. Long will send an invitation and confirm TA participation before PA1 submission. Record decisions in GitHub issues or meeting records.
GitHub repository	The agreed workflow is issue → branch → PR → at least one peer approval → merge. No PR has been opened, so there is no PR-review evidence. Main protection and required approval are unverified, and instructors/TA do not yet have repo access.
GitHub Projects	The active board lists all current WBS work packages with owners and deadlines. Update status, effort, dependencies, and evidence as work proceeds through the `Backlog` → `Ready` → `In Progress` → `In Review` → `Done` workflow.
Prism	Collaborate on one shared report and compile review PDFs; arrange project access and assign an exporter, then export source manually at PA review milestones.
CI	Initial CI setup is planned for W8; add lint, unit-test, and coverage checks as implementation proceeds and course requirements are clarified. No active CI or passing checks are claimed.
Weekly status report	The team lead/PM compiles the weekly status report and commits it to `docs/management/weekly/` by 23:59 every Sunday (`Asia/Ho_Chi_Minh`), from week 2 through week 10. Hold the weekly status meeting for 15 minutes; agree the TA meeting schedule separately.

7.2 Repository Structure

All paths below are relative to the repository root.

Folder	Content
`src/`	Room4U application source; keep the editable application code in this single working location.
`docs/management/`	Project plan, WBS, risk register, decisions, and meeting records.
`docs/management/weekly/`	Weekly status reports, one for each reporting week from week 2 through week 10.
`docs/requirements/`	Vision, SRS, use cases, and mockups.
`docs/design/`	Architecture (C4), ADRs, UML models, and UI design.
`docs/test/`	Test plan, test cases, test results, and test reports.
`docs/ai-usage/`	AI usage logs.
`pa/PA0/`–`pa/PA5/`	One release folder per PA, containing that submission's exported source and final PDF as a matched pair.

The shared Prism project is the report's only editable master. Working management, requirements, design, and test documents belong under `docs/`; each PA folder holds only its reviewed source export and accepted PDF, not a second working copy.

7.3 Task Workflow

1. Create an issue with acceptance criteria and evidence links; assign an owner, deadline, effort, and dependencies.
2. Move the task to `Ready`, create a separate branch from `main`, and implement the change.
3. Commit using Conventional Commits (for example, `feat:`, `fix:`, `docs:`, or `test:`).
4. Open a PR linked to the issue and obtain at least one peer approval before merge. For documentation, review the content, compile LaTeX, and inspect the PDF; for application code, run configured lint, unit tests, and any agreed coverage checks.
5. Merge only after the required peer approval and applicable checks pass, then mark the issue `Done` and attach the completed work or other evidence.

Never commit `.env` files, passwords, API keys, or service-account credentials.

8 AI Usage Log

#	Member	Tool	Task / purpose	How the output was verified and changed
1	Trần Thiên Phúc	ChatGPT	Drafted Risk Management content and proposed risks, Probability/Loss scores, owners, mitigations, and contingencies.	Cross-checked the full section and RE calculations against the lecture slides, assessed risk and response suitability, selected the final content, and take responsibility for the submitted version.
2	Trần Thiên Phúc	ChatGPT	Drafted tool descriptions, repository structure, and the issue–branch–PR–review workflow.	Reviewed all content and checked that confirmed arrangements were distinguished from planned settings; I selected the material used and take responsibility for the final text.
3	Lại Minh Thông	Claude	Translated the content of Sections 4–5.2 (Proposed Solution, Process and Phase Gates, Work Breakdown Structure) from Vietnamese into English, polished wording and consistency, and helped write the LaTeX source (tables, WBS, and C4 context diagram).	Re-read the translated sections, compared them with the original Vietnamese content, checked spelling, LaTeX font, and compile errors, reviewed the final text, and take responsibility for it.
4	Nguyễn Thành Long	ChatGPT	Drafted the Vision section and translated the completed section into English.	Reviewed all content for correctness and consistency with the project idea; I take responsibility for the final text.
5	Nguyễn Kim Ngân	ChatGPT	Used ChatGPT to help with effort estimation, project scheduling, team roles and responsibilities, and cost estimation.	Reviewed the suggestions, checked the estimates and calculations, and adjusted the content to match the team's WBS and project plan.